TALKING TO THE USER

Input, and doing something with it

Reading input

`std::print` sends text to the console.

`std::cin` / `std::getline` read text back from it.

```
std::string name;
std::print("What is your name? ");
std::getline(std::cin, name); // reads a full line, including spaces
```

`>>` vs. `getline`

```
int age{};
std::print("How old are you? ");
std::cin >> age;
```

`>>` reads a single value and **stops at whitespace** - different from `getline`, which reads the whole line, spaces included.

DOING MORE

It's not just echoing back

Let's **do something** with the data with it.

```
int birthYear{};
std::print("What year were you born? ");
std::cin >> birthYear;

int turns100In = birthYear + 100;
std::println("You'll turn 100 in the year {}.", turns100In);
```

FULL PROGRAM

Where we are now

```
std::print("What is your name? ");
std::getline(std::cin, name);

std::print("How old are you? ");
std::cin >> age;

std::print("What year were you born? ");
std::cin >> birthYear;

greetPerson(name);
std::println("You are {} years old.", age);

int turns100In = birthYear + 100;
std::println("You'll turn 100 in the year {}.", turns100In);
```

Debugging a read

You can't step **into** `std::cin >> age` to watch it read - it's atomic as far as the debugger sees it.

> The pattern: breakpoint on the line **after** the read, step over the read itself, then inspect the variable that just got filled in.